

Discord Clone

A Project Report Submitted by:

N Venkata Praneeth – AP23110010960

P Venkata Sravan – AP23110010963

M Gnaneshwar – AP23110010937

V Yashwanth – AP23110010967

Under the Supervision of

Mr. Himanshu Mishra

Assistant Professor

Department of Computer Science and Engineering

SRM University-AP

In partial fulfilment for the requirements of the

FullStack Development-II(SEC 160) Project

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



SRM
UNIVERSITY AP
— **Andhra Pradesh**

CERTIFICATE

This is to certify that the project work entitled “**Discord Clone**” is a Bonafide record of project work carried out by the following students:

- **Mr. N. Venkata Praneeth** (AP23110010960)
- **Mr. P. Venkata Sravan** (AP23110010963)
- **Mr. M. Gnaneshwar** (AP23110010937)
- **Mr. V. Yashwanth** (AP23110010967)

from the **Department of Computer Science and Engineering, SRM University-AP.**

The students conducted this project work under my supervision during the period January 2026 to April 2026. It is further certified that, to the best of my knowledge, this project has not previously formed the basis for the award of any degree or any similar title to this or any other candidate.

This is also to certify that the project work represents the teamwork of the candidates.

Station: Mangalagiri

Date: 06/12/2025

Mr. Himanshu Mishra

Assistant Professor

Department of Computer Science & Engineering

SRM University-AP

Andhra Pradesh.

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

We are extremely grateful and express my profound gratitude and indebtedness to our project guide, **Mr. Himanshu Mishra**, Department of Computer Science & Engineering, SRM University, Andhra pradesh, for his kind help and for giving us the necessary guidance and valuable suggestions in completing this project work.

TABLE OF CONTENTS

Certificate.....	2
Acknowledgement.....	3
1. Introduction.....	5
2. Scenario Based Case Study.....	6
3. System Requirements.....	7-8
4. Project Architecture.....	9-10
5. ER Diagrams.....	11-13
6. Key Features.....	13-14
7. Roles and Responsibilities.....	15-17
8. MVC Pattern.....	18-19
9. Project Setup and Configuration.....	20-33
9.1 Backend Development.....	22-25
9.2 Database Management.....	26-30
9.3 Frontend Development.....	31-33
10. Output Screenshots.....	34-39
11. Demo Video and Links.....	40

1. INTRODUCTION

The **Discord Clone** is a real-time communication web application designed to enable users to interact efficiently through text-based messaging and server-based communication. This application replicates the core functionalities of Discord, allowing users to create servers, join channels, and communicate with others in an organized and user-friendly environment.

In today's digital world, communication platforms play a crucial role in connecting people for various purposes such as education, teamwork, and social interaction. The Discord Clone provides a structured platform where users can create communities (servers), divide conversations into channels, and exchange messages instantly. This helps in maintaining organized discussions and improves collaboration among users.

The application is developed using modern web technologies to ensure high performance and scalability. The frontend is built using **React.js**, which provides a responsive and dynamic user interface. The backend is implemented using **Node.js** and **Express.js**, which handle server-side logic and API requests efficiently. The application uses **MongoDB** as the database to store user data, messages, and server information in a flexible and scalable manner.

One of the key objectives of this project is to demonstrate the implementation of full-stack web development concepts such as RESTful APIs, database integration, authentication, and real-time data handling. The system ensures secure user authentication and smooth communication between frontend and backend components.

Overall, the Discord Clone serves as a practical example of a modern communication platform, showcasing how real-time applications can be built using the MERN stack (MongoDB, Express.js, React.js, Node.js). It provides an efficient, scalable, and user-friendly solution for online communication and collaboration.

2. SCENARIO BASED CASE STUDY

Consider a college student named Rahul who is working on multiple academic projects and group assignments. Rahul often needs to communicate with his classmates, share ideas, discuss tasks, and collaborate effectively. However, using traditional communication methods like email or basic messaging apps becomes inefficient, as conversations get mixed up and important information is difficult to track.

To overcome this challenge, Rahul starts using the **Discord Clone** application. He creates a dedicated server for his project team and organizes different channels such as “General Discussion,” “Project Updates,” and “Task Allocation.” Each channel serves a specific purpose, helping the team maintain structured and focused communication.

Using the application, Rahul and his teammates can send messages in real time, share updates instantly, and keep track of ongoing discussions without confusion. The server-based structure allows them to separate conversations based on topics, making collaboration more efficient and organized.

Additionally, Rahul can invite new members to the server whenever needed, ensuring smooth onboarding of teammates. The system also allows users to manage their profiles securely and interact within a controlled environment.

Through the use of the Discord Clone, Rahul experiences improved productivity, better communication, and enhanced teamwork. The application helps eliminate confusion, reduces communication delays, and ensures that all team members stay connected and informed.

This scenario highlights how the Discord Clone application can be effectively used in real-world situations such as academic collaboration, team coordination, and online community building.

3. SYSTEM REQUIREMENTS

To ensure the smooth development, deployment, and execution of the Discord Clone application, certain system requirements must be met. These requirements include both software and hardware specifications necessary for efficient performance.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux – Supports cross-platform development and testing.
- **Node.js (v16 or above):** Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing. [Download Node.js](#)
- **npm (v8 or above):** A package manager required to install dependencies for React and related libraries.
- **React.js:** A JavaScript library for building dynamic and responsive user interfaces.
- **Browser:** Google Chrome / Firefox (latest version) – For rendering and testing the UI in real-time.
- **Express.js:** A lightweight web framework for building RESTful APIs.
- **MongoDB:** A NoSQL database used to store structured and unstructured data related to farms, users, and resources. [Download MongoDB](#)
- **Postman:** Tool for testing APIs during development.

- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.

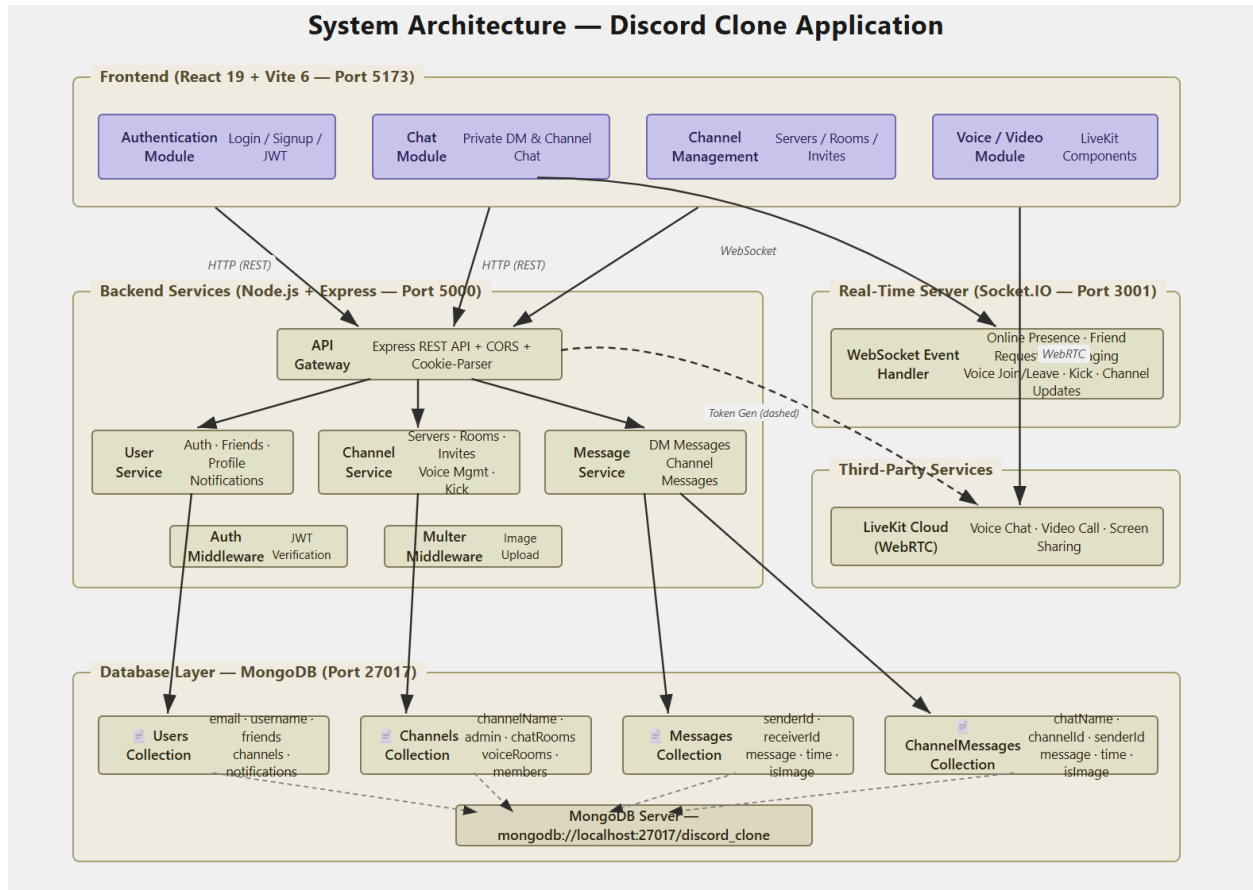
2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage:** At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- **Display:** 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

4. PROJECT ARCHITECTURE

TECHNICAL ARCHITECTURE



The Discord Clone application follows a structured client-server architecture where the interaction between different components ensures smooth communication and data flow.

The process begins with the **user**, who interacts with the application through the frontend interface. The frontend of the application is developed using **React.js**, which provides a dynamic and responsive user experience.

When a user performs any action such as sending a message, creating a server, or joining a channel, the frontend sends a request to the backend server. The backend is developed using **Node.js** and **Express.js**, which handle all the application logic and API requests.

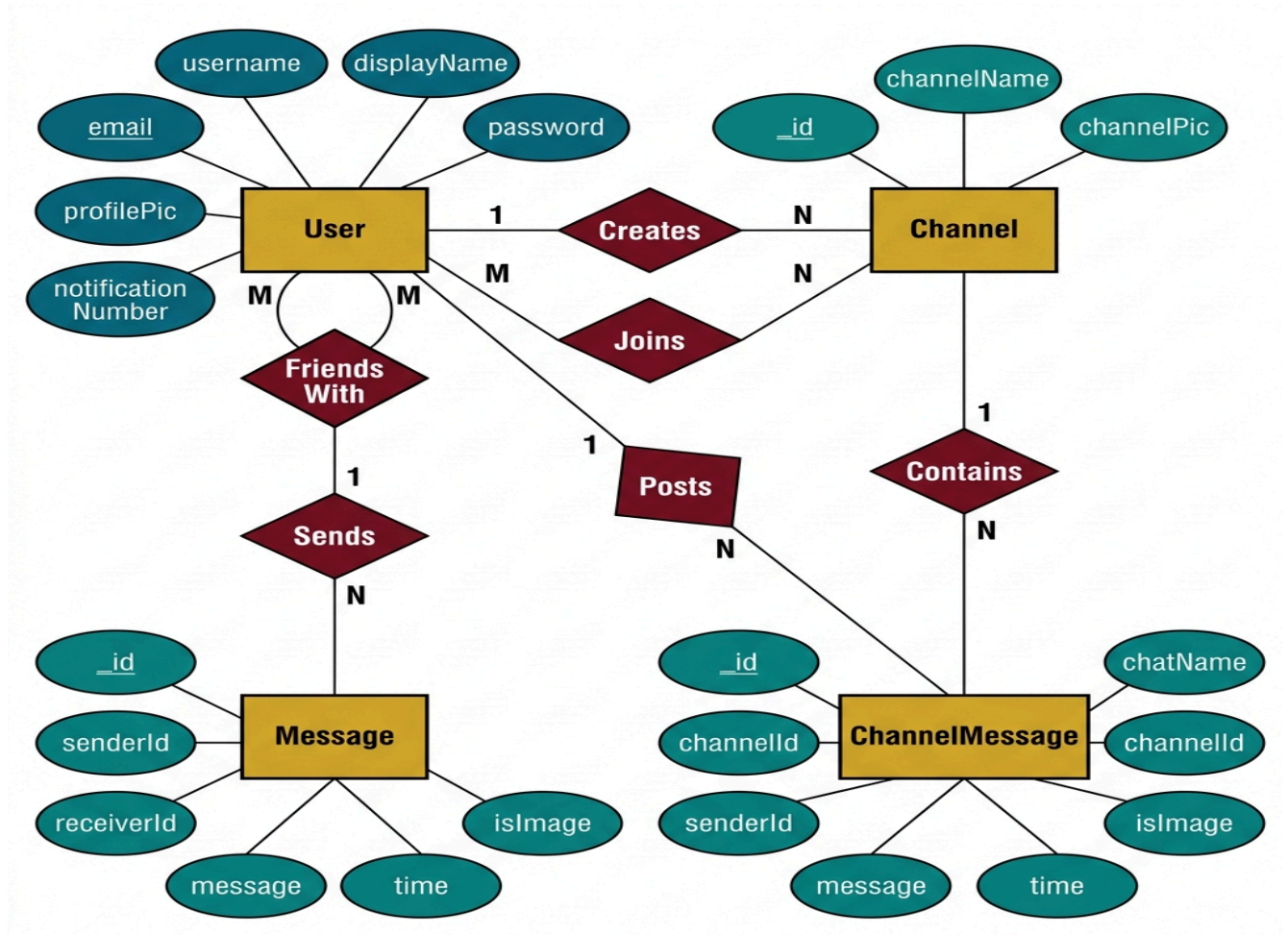
The backend processes the request and communicates with the **MongoDB database**, where all the data such as user details, messages, servers, and channels are stored. MongoDB provides a flexible and scalable way to manage data efficiently.

Once the backend processes the request and retrieves or updates the required data, it sends a response back to the frontend. The frontend then displays the updated information to the user in real time.

This architecture ensures:

- Efficient data flow
- Scalability of the application
- Separation of frontend and backend logic
- Better performance and maintainability

5. ER DIAGRAM



The Entity-Relationship (ER) diagram of the Discord Clone application represents the logical structure of the database and illustrates how different entities are related to each other within the system. The ER model helps in organizing data efficiently and ensures smooth communication between various components of the application.

The primary entities in this system include **User**, **Server**, **Channel**, and **Message**, each playing a crucial role in enabling real-time communication.

1. User Entity

The **User** entity represents individuals who access and interact with the application. Each user is uniquely identified by a user ID and contains attributes

such as username, email, and password. Users can create servers, join existing servers, and send messages within channels.

Users are the core participants of the system, and all major activities such as communication, server creation, and interaction are initiated by them.

2. Server Entity

The **Server** entity represents a group or community created by users. Each server is uniquely identified by a server ID and contains attributes such as server name and owner ID. A user who creates a server becomes its owner and has control over its management.

Servers act as containers for communication, allowing multiple users to interact within a shared environment.

3. Channel Entity

The **Channel** entity represents subdivisions within a server that help organize communication. Each channel is identified by a channel ID and contains attributes such as channel name and server ID.

Channels allow users to separate conversations based on topics such as general discussion, announcements, or specific project-related communication.

4. Message Entity

The **Message** entity represents the communication exchanged between users. Each message contains attributes such as message ID, content, timestamp, sender ID, and channel ID.

Messages are stored within channels and enable real-time interaction among users. This entity is essential for maintaining conversation history and ensuring continuity in communication.

Relationships Between Entities

The relationships between these entities define how the system operates:

- A **User can create multiple Servers**, establishing a one-to-many relationship.
- A **Server can contain multiple Channels**, allowing structured communication within groups.
- A **Channel can store multiple Messages**, representing ongoing conversations.
- A **User can send multiple Messages**, linking users to communication activities.

These relationships ensure efficient data flow and organization within the application.

The ER diagram provides a clear understanding of how data is structured and interconnected in the Discord Clone application. It plays a vital role in designing the database schema and helps developers maintain consistency, scalability, and efficiency in handling user interactions and real-time communication.

6. KEY FEATURES

The Discord Clone application provides several important features that enhance user communication and ensure a smooth and efficient experience. These features are designed to support real-time interaction, organized discussions, and secure access to the platform.

User Authentication and Authorization

The application includes a secure authentication system that allows users to register and log in using their credentials. This ensures that only authorized users can access the platform and perform actions within the system, maintaining security and data integrity.

Server Creation and Management

Users have the ability to create and manage servers, which act as dedicated

spaces for communication. Each server can be customized and controlled by its owner, allowing efficient organization of users and discussions.

Channel-Based Communication

Within each server, multiple channels can be created to organize conversations based on specific topics. This helps in maintaining structured communication and avoids confusion by separating discussions into different categories.

Real-Time Messaging

The application supports real-time messaging, enabling users to send and receive messages instantly. This ensures seamless communication without delays, making the platform suitable for collaboration and interactive discussions.

User-Friendly Interface

The frontend is designed using React.js, providing a responsive and intuitive user interface. Users can easily navigate between servers, channels, and messages, enhancing overall usability.

Message Storage and History

All messages are stored in the database, allowing users to access previous conversations at any time. This ensures continuity in communication and helps users refer back to important discussions.

Scalable Architecture

The application is built using the MERN stack, which ensures scalability and efficient handling of multiple users and large volumes of data. This makes the system reliable for future expansion.

Secure Data Handling

The system implements proper validation and secure data storage practices to protect user information and communication. This ensures privacy and reliability within the platform.

7. ROLES AND RESPONSIBILITIES

The Discord Clone application defines specific roles to ensure proper usage, management, and smooth functioning of the system. Each role has its own set of responsibilities that contribute to efficient communication and system reliability.

User

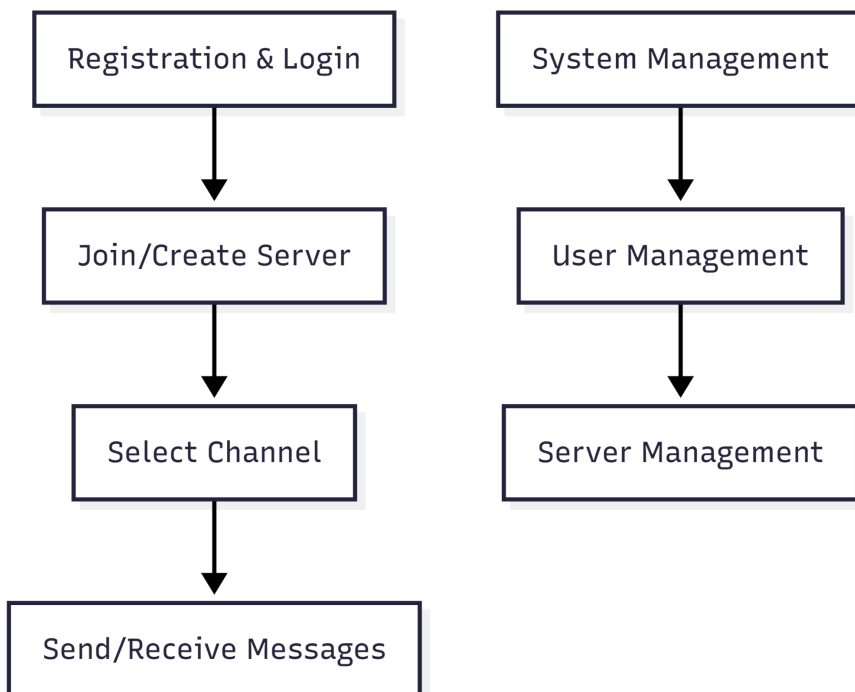
- **Registration and Account Management:** Users are responsible for creating an account by providing accurate personal information and managing their login credentials securely. They can update their profile details and ensure the confidentiality of their account information.
- **Server and Channel Interaction:** Users can create new servers or join existing ones. They are responsible for participating in different channels within servers and maintaining structured and meaningful communication.
- **Messaging and Communication:** Users can send and receive messages in real time. They should communicate responsibly and follow proper guidelines while interacting with other users within the platform.
- **Content Responsibility:** Users are responsible for the content they share within servers and channels. They must ensure that their messages are appropriate and do not violate any rules or policies.
- **Collaboration and Engagement:** Users actively participate in discussions, collaborate with others, and contribute to productive communication within the platform.
- **Feedback and Reporting:** Users can provide feedback or report issues related to the system, helping improve the overall application performance and user experience.

Admin

- **System Management:** The admin is responsible for managing the overall system performance and ensuring smooth operation of the application. This includes monitoring backend services and maintaining system stability.

- **User Management:** The admin manages user accounts, including handling authentication issues, monitoring user activities, and taking necessary actions such as account restriction if required.
- **Server and Channel Management:** The admin oversees the creation and management of servers and channels, ensuring proper organization and efficient communication within the system.
- **Data Management:** The admin ensures that all data stored in the database is consistent, secure, and properly maintained. They are responsible for managing data integrity and handling backups if necessary.
- **Security and Maintenance:** The admin ensures that the system is secure from unauthorized access and vulnerabilities. Regular updates, bug fixes, and maintenance tasks are handled to keep the system reliable.
- **Monitoring and Support:** The admin monitors system performance and provides support in case of technical issues. They ensure that users experience minimal disruptions while using the platform.

User Flow & Admin



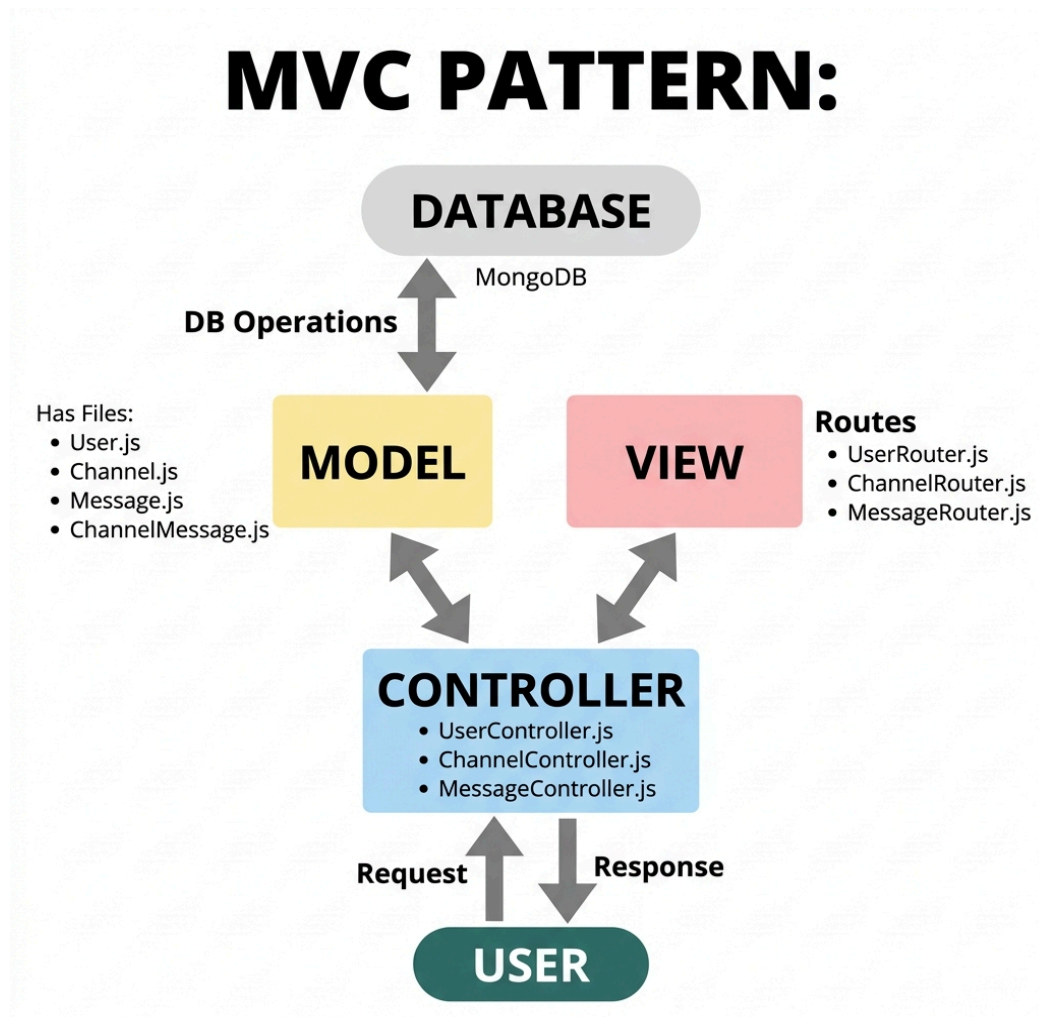
User Flow

- **Registration & Login** – Users create an account and securely log in to access the application.
- **Join/Create Server** – Users can create new servers or join existing ones to start communication.
- **Channel Selection** – Users navigate through different channels within a server based on topics.
- **Messaging** – Users send and receive messages in real time within selected channels.

Admin Flow

- **System Management** – Admin monitors the overall performance and ensures smooth functioning of the system.
- **User Management** – Admin manages user accounts, including authentication and access control.
- **Server Management** – Admin oversees servers and ensures proper organization and maintenance.

8. MVC PATTERN



The Discord Clone application follows the Model-View-Controller (MVC) architectural pattern, which is a software design approach that separates the application into three interconnected components. This separation improves scalability, maintainability, and organization of the application.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related operations in the application. It defines the structure of the data and interacts with the database. In this project, MongoDB is used to store data such as user details, servers, channels, and messages. The models are implemented using schema-based structures, ensuring efficient data management and retrieval.

View Layer (Frontend Layer)

The View layer represents the user interface of the application. It is responsible for displaying data to the user and capturing user interactions. In this project, the frontend is developed using React.js, which provides a dynamic and responsive interface. The View layer communicates with the Controller to send and receive data.

Controller Layer (Logic Layer)

The Controller layer acts as an intermediary between the Model and View layers. It processes incoming requests from the frontend, applies business logic, and interacts with the Model to fetch or update data. After processing, it sends the appropriate response back to the View layer.

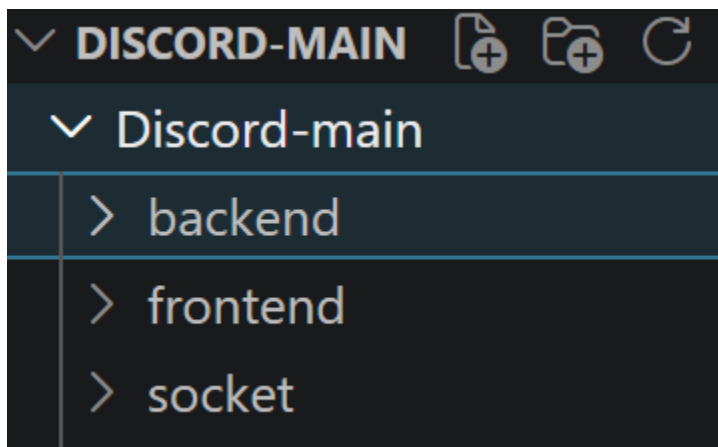
Advantages of MVC Architecture

- **Separation of Concerns:** Each layer has a specific responsibility, making the application easier to manage and understand.
- **Scalability:** New features can be added without affecting other components of the system.
- **Maintainability:** Code is organized into separate modules, making debugging and updates easier.
- **Reusability:** Components can be reused across different parts of the application.
- **Efficient Development:** Multiple developers can work on different layers simultaneously without conflicts.

9. PROJECT SETUP AND CONFIGURATION

Creating project folder

1. Create a new folder with your project name **Discord Clone**.
2. Inside that folder create three new folders.
3. Name one as **frontend**.
4. Name another one as **backend**.
5. Name another one as **socket**.
6. Now open that folder in VS Code.



Client setup (installing react app)

- Open the **frontend folder** in the terminal of VS Code.
 - `npm create vite@latest . -- --template react`
- Select React framework from the given options.
 - Select a framework:
 - | React
- Select JavaScript variant from the given options.
 - Select a variant:
 - | JavaScript

- Now let's navigate to the frontend folder by giving the following command.
 - `cd frontend`
- To install all the packages run the following command.
 - `npm install`
- To start the React server type the following command.
 - `npm run dev`

Server setup (npm init)

- Open **backend folder** in terminal of VS Code.
 - `npm init -y`
- Install required packages:
 - `npm install express mongoose cors dotenv`
- Create files:
 - `index.js`
- Create folders:
 - `controller`
 - `model`
 - `router`
 - `middleware`

Socket setup

- Open **socket folder** in terminal of VS Code.
 - `npm install`
- Create files:
 - `index.js`

- To run socket server:
 - node [index.js](#)

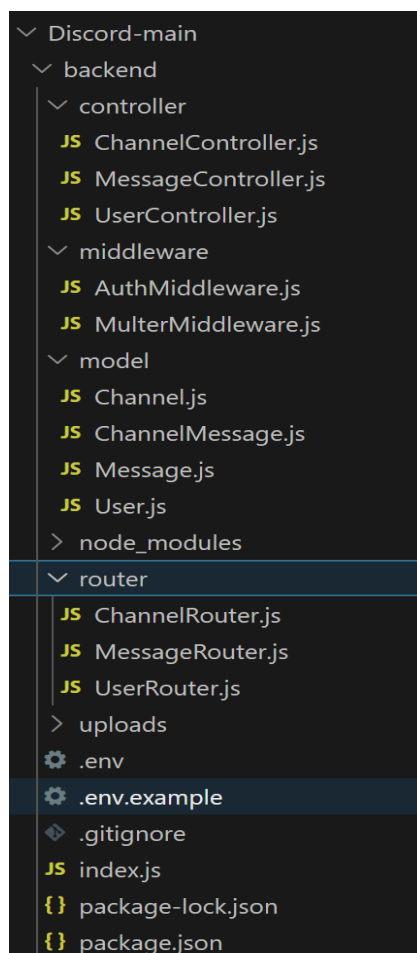
9.1 Backend Development

The backend of the Discord Clone application is developed using Node.js and Express.js. It is responsible for handling API requests, managing data, and connecting to the database.

Backend Reference Video:

<https://drive.google.com/file/d/1Tvm1PSyX2fl0oTSFowD7lGnZ7BLQrqb/view?usp=sharing>

Backend Structure:



Controllers

- **UserController.js** → Handles user-related operations such as registration, login, and authentication.
- **MessageController.js** → Handles sending and receiving messages between users.
- **ChannelController.js** → Handles creation and management of channels.

db

- **config.js** → Contains database connection setup using MongoDB.

middleware

- **AuthMiddleware.js** → Middleware for authentication and authorization using JWT.
- **MulterMiddleware.js** → Middleware for handling file uploads such as images.

models

- **User**
 - User.js → Schema/model for storing user details.
- **Message**
 - Message.js → Schema/model for storing messages.
- **Channel**
 - Channel.js → Schema/model for managing channels.
- **ChannelMessage**
 - ChannelMessage.js → Schema/model for storing channel messages.

routes

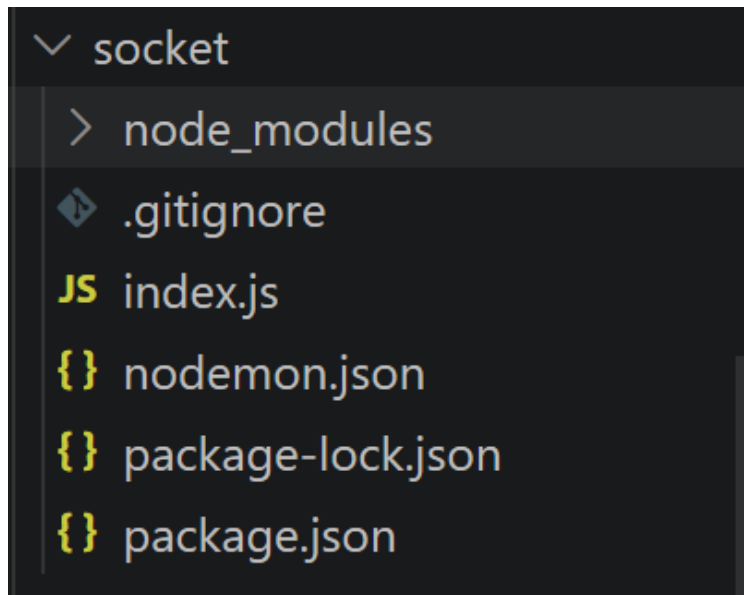
- **UserRouter.js** → Defines API routes for user operations (register, login, etc.).
- **MessageRouter.js** → Defines API routes for messaging.
- **ChannelRouter.js** → Defines API routes for channel operations.

Socket setup

Open socket folder in terminal of VS Code.

Run the following command: *npm install*

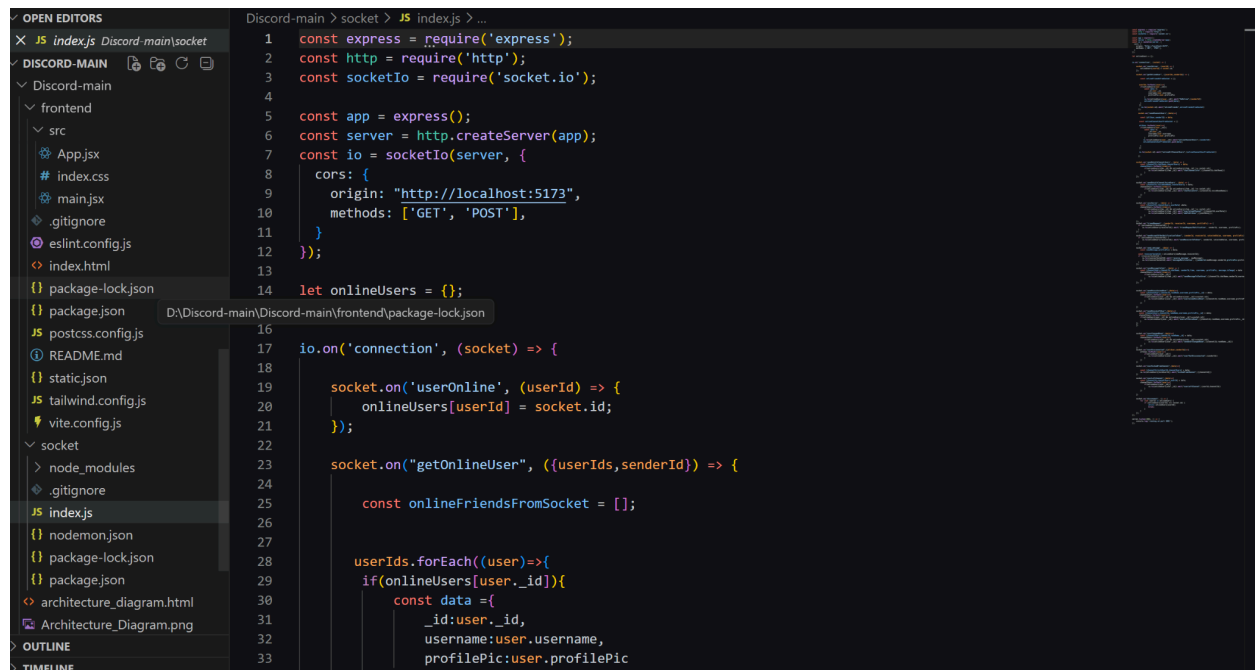
To start the socket server: *node [index.js](#)*



Socket Setup

The socket module is used to handle real-time communication between users using WebSocket.

- **index.js** → Main file responsible for handling socket connections and real-time messaging.



9.2 DATABASE DEVELOPMENT

Configure MongoDB:

- Install Mongoose. In your Node.js project directory, run: `npm install mongoose`
- Create database connection.
- Create Schemas & Models.

Create Database connection

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Discord-main > backend > JS index.js > ...

```
1  const express = require("express");
2  const app = express();
3  const mongoose = require("mongoose");
4  const dotenv = require("dotenv");
5  const cors = require("cors");
6  const UserRoutes = require("./router/UserRouter");
7  const ChannelRoutes = require("./router/ChannelRouter");
8  const MessageRoutes = require("./router/MessageRouter");
9  const cookieParser = require("cookie-parser");
10 const path = require("path");
11
12 const { AccessToken } = require("livekit-server-sdk");
13 dotenv.config();
14
```

```
const connectToDB = async()=>{
  await mongoose.connect(process.env.MONGO_URL);
  console.log("connected to DB");
}

app.listen(process.env.PORT, async()=>{
  await connectToDB();
  console.log(`server listening on port ${process.env.PORT}`);
})
```

Create Schemas

Firstly, configure the schemas for MongoDB database to store data in a structured format. The schemas are designed based on the ER diagram of the system.

User Model

The User model is used to store user information such as username, email, and password.

User.js:

```
d-main > backend > model > JS User.js > ...
const mongoose = require("mongoose");

const UserSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true
  },
  displayName: {
    type: String,
    required: true,
  },
  username: {
    type: String,
    required: true,
    unique: true
  },
  password: {
    type: String,
    required: true
  },
  profilePic: { type: String, default: "" },
  ownChannel: [
    {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Channel",
      default: null
    }
  ],
  joinedChannel: [
    {
      type: mongoose.Schema.Types.ObjectId,
```

Message Model

The Message model is used to store messages sent by users including content, sender, and timestamp.

Message.js:

```
const mongoose = require("mongoose");

const MessageSchema = new mongoose.Schema({
  senderId: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "User"
  },
  receiverId: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "User"
  },
  message: { type: String, required: true },
  time: {
    type: String,
    required: true
  },
  isImage: {
    type: Boolean,
    required: true
  },
}, { timestamps: true })

module.exports = mongoose.model("Message", MessageSchema)
```

Channel Model

The Channel model is used to manage different channels within servers.

Channel.js:

```
const mongoose = require("mongoose");

const ChannelSchema = new mongoose.Schema({
  channelName: {
    type: String,
    required: true
  },
  admin: [
    {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
    }
  ],
  chatChannel: [
    {
      roomName: {
        type: String,
        required: true,
      },
      messages: [
        {
          sender: {
            type: String,
            required: true,
          },
          content: {
            type: String,
            required: true,
          },
          timestamp: {
            type: Date,
            default: Date.now,
          }
        }
      ]
    }
  ]
});
```

```
const ChannelSchema = new mongoose.Schema({
  voiceChannel: [
    {
      voiceRoomName: {
        type: String,
        default: null
      },
      voiceUsers: [
        {
          type: mongoose.Schema.Types.ObjectId,
          ref: "User",
          default: null
        }
      ]
    }
  ],
  channelUsers: [
    {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      default: null
    }
  ],
  channelPic: {
    type: String,
    default: ""
  }
});

module.exports = mongoose.model("Channel", ChannelSchema);
```

Channel Message Model

This model is used to store messages inside channels.

ChannelMessage.js:

```
const mongoose = require("mongoose");

const ChannelMessageSchema = new mongoose.Schema({
  chatName:{
    type:String,
    required:true
  },
  channelId:{
    type:mongoose.Schema.Types.ObjectId,
    required:true,
    ref:"Channel"
  },
  senderId:{
    type:mongoose.Schema.Types.ObjectId,
    required:true,
    ref:"User",
  },
  message: { type: String, required: true },
  time:{
    type:String,
    required:true
  },
  isImage:{
    type:Boolean,
    required:true
  },
},{ timestamps: true },

)

module.exports = mongoose.model("ChannelMessage",ChannelMessageSchema);
```

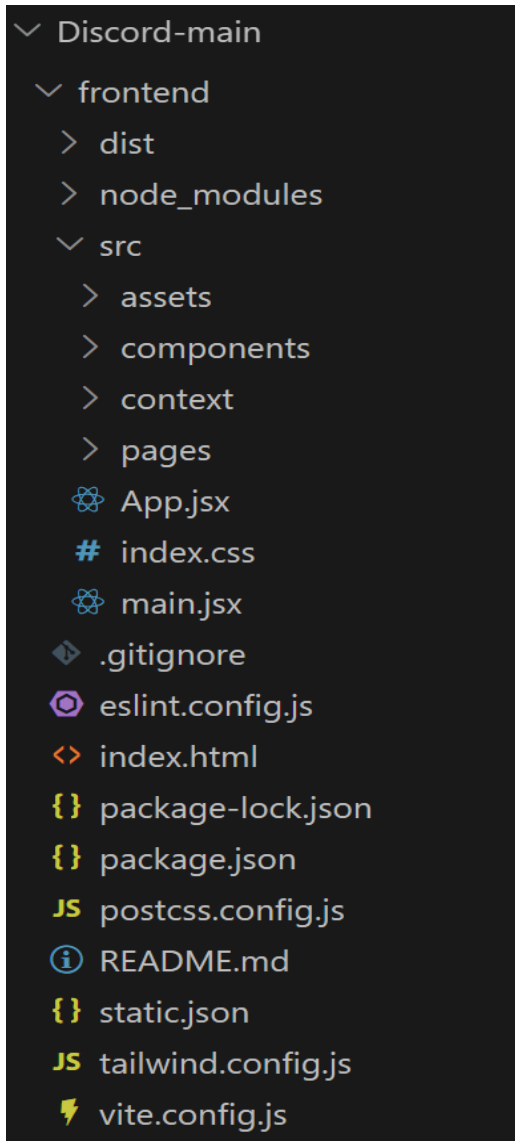
9.3 Front-end Development

The frontend of the Discord Clone application is developed using React.js. It provides an interactive and user-friendly interface for users to communicate in real time.

Frontend Reference Video:

<https://drive.google.com/file/d/1vxXOYjp05K9d4xmd8Cfb59S7V2qgsXpS/view?usp=sharing>

Frontend Structure:



Pages (src/pages)

- **Login.jsx** → Handles user login with authentication.
- **Signup.jsx** → Allows users to register with validation.
- **Home.jsx** → Displays dashboard with friends and navigation.
- **Channel.jsx** → Displays server channels and chat interface.
- **FriendChat.jsx** → Handles private messaging between users.
- **Profile.jsx** → Allows users to manage profile details.

Components (src/components)

Navigation & Layout

- **Sidebar.jsx** → Displays servers and navigation icons.
- **Menu.jsx** → Displays direct messages and navigation options.
- **ChannelMenu.jsx** → Displays channel list inside a server.
- **TopBar.jsx** → Displays navigation tabs for friends section.
- **BottomProfile.jsx** → Displays user profile and controls.

Chat Components

- **ChatArea.jsx** → Displays messages in real time.
- **ChatComplement.jsx** → Displays individual message structure.
- **PrivateChat.jsx** → Displays private messages between users.
- **ChatRightArea.jsx** → Displays active users in a channel.

Friend Management

- **HomeFriend.jsx** → Displays friend list item.
- **MenuFriends.jsx** → Displays friend in sidebar messages.
- **AddFriendMenu.jsx** → Allows sending friend requests.
- **PendingFriend.jsx** → Displays pending requests.
- **FriendNotification.jsx** → Shows message notifications.

Server/Channel Management

- **Server.jsx** → Displays server icon in sidebar.
- **CreateChannel.jsx** → Allows creating new server.
- **CreateRoom.jsx** → Creates text chat rooms.
- **CreateVoiceRoom.jsx** → Creates voice rooms.
- **ChannelGeneralSettingsArea.jsx** → Handles server settings.
- **ChannelMember.jsx** → Displays server members.

Voice/Video Components

- **ChannelVoiceItem.jsx** → Displays voice room and user count.
- **VoiceComponent.jsx** → Displays active users in voice chat.
- **ChannelUserControlArea.jsx** → Provides voice controls.
- **VideoConferenceRoom.jsx** → Handles video conferencing.

Context (src/context)

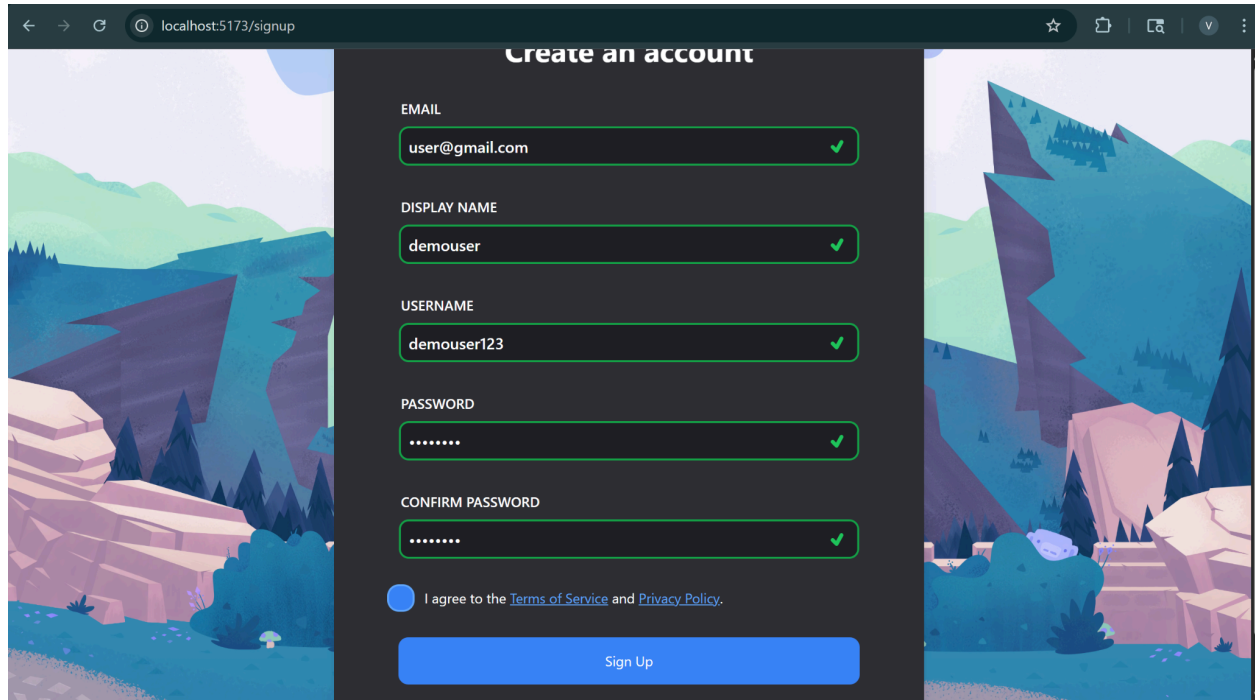
- **UserContext.jsx** → Manages global state, user session, and socket connection.

Main Files

- **App.jsx** → Handles routing and main application flow.
- **main.jsx** → Entry point of the application.

10. OUTPUT SCREENSHOTS

User's Signup Page:

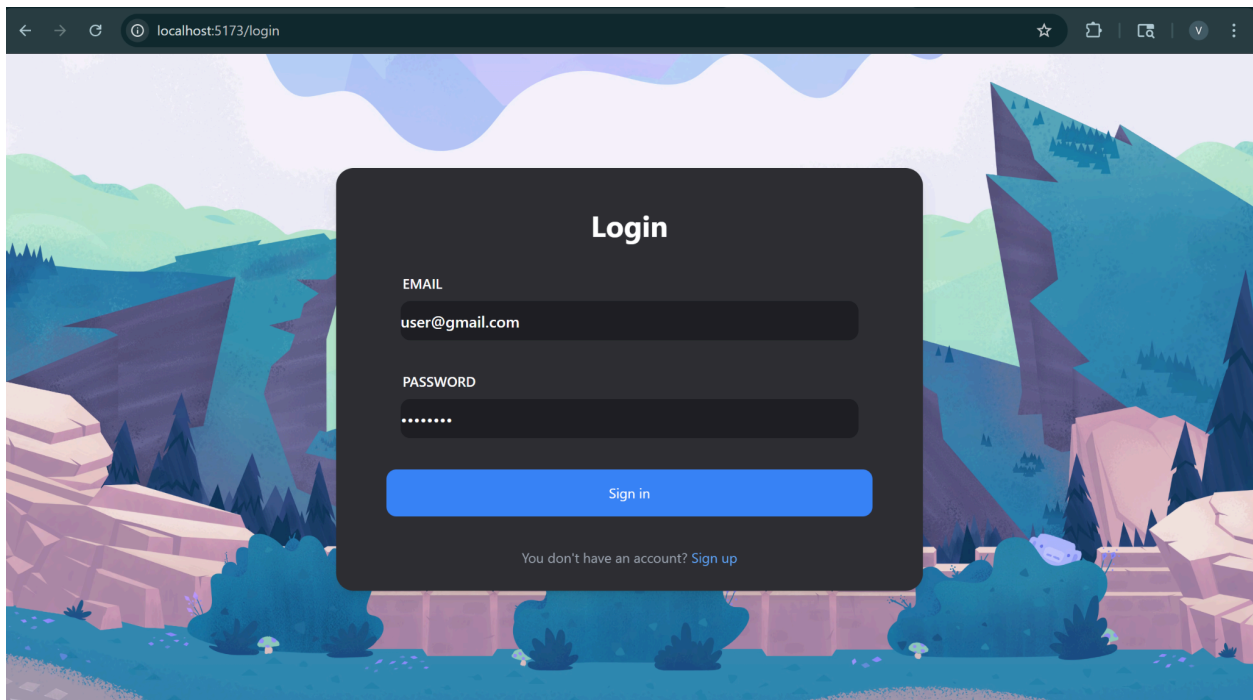


The screenshot shows a web browser window with the address bar displaying 'localhost:5173/signup'. The page has a dark theme and a stylized landscape background with mountains and trees. The main heading is 'Create an account'. Below it, there are five input fields, each with a green checkmark indicating successful validation:

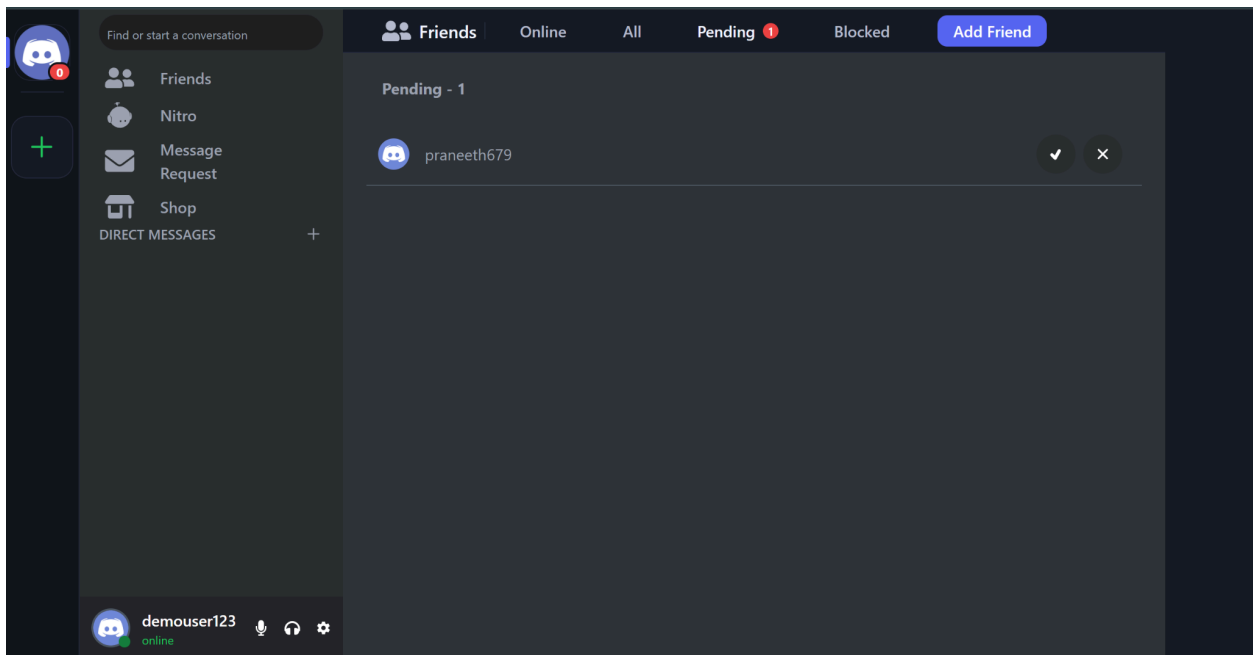
- EMAIL: user@gmail.com
- DISPLAY NAME: demouser
- USERNAME: demouser123
- PASSWORD: (masked with dots)
- CONFIRM PASSWORD: (masked with dots)

Below the input fields, there is a checkbox labeled 'I agree to the [Terms of Service](#) and [Privacy Policy](#).' which is currently unchecked. At the bottom, there is a blue button labeled 'Sign Up'.

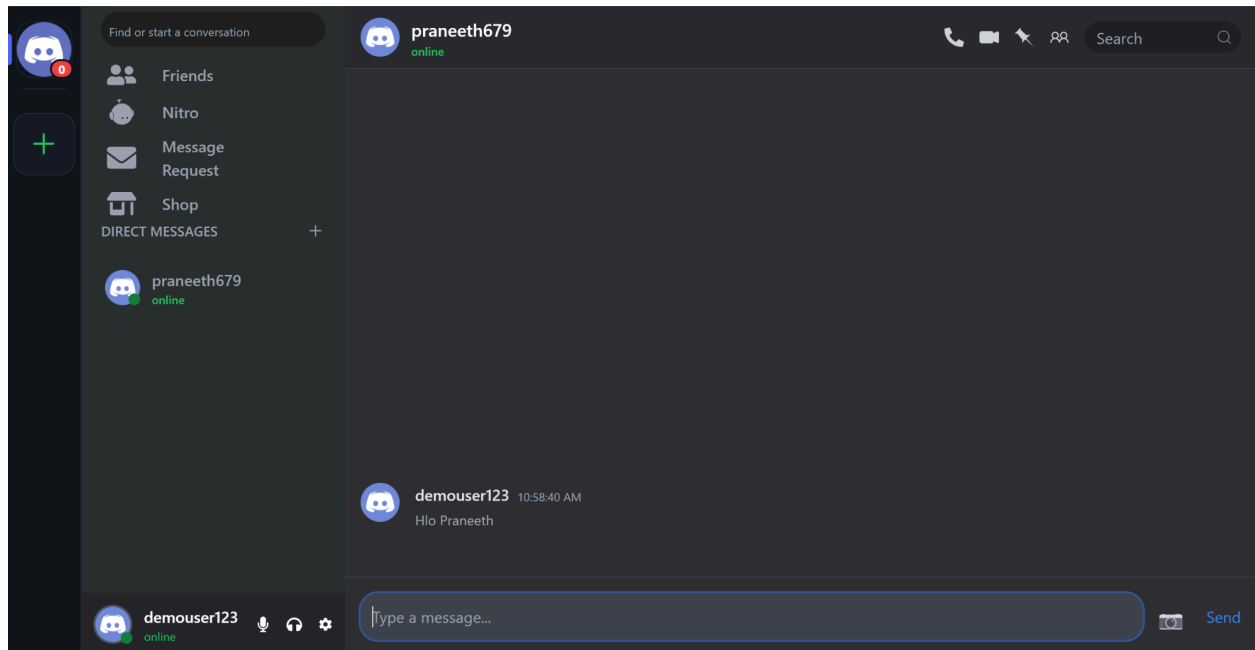
User's Login Page:



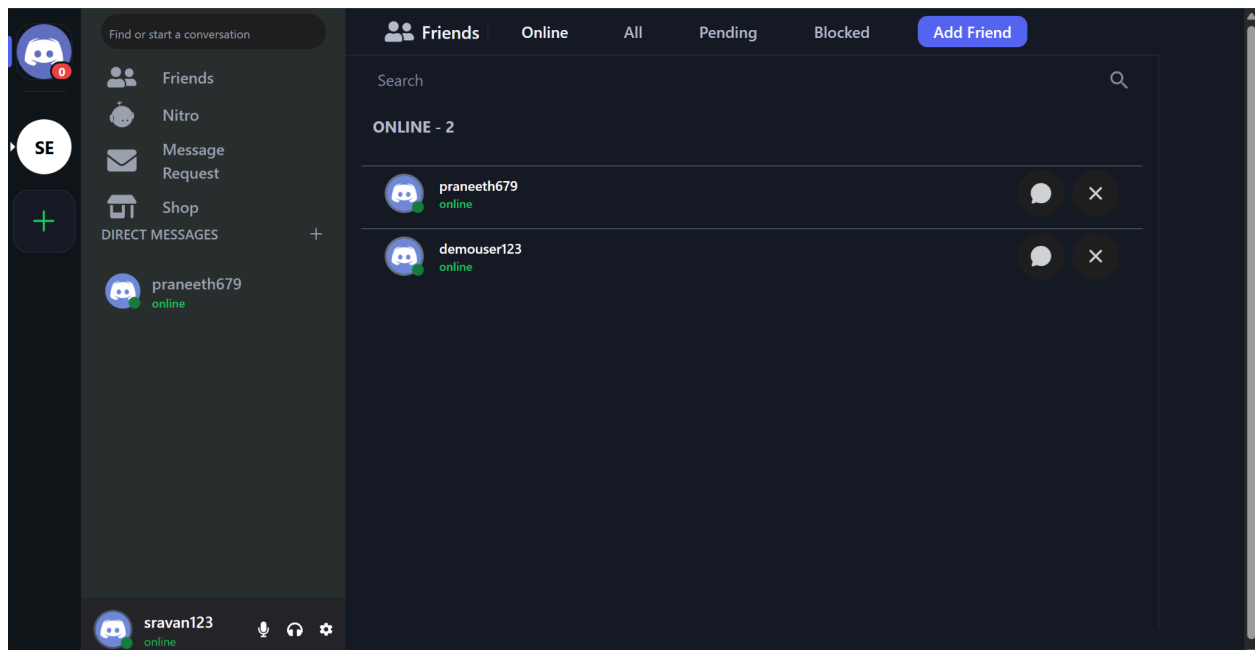
Pending Friend Requests:



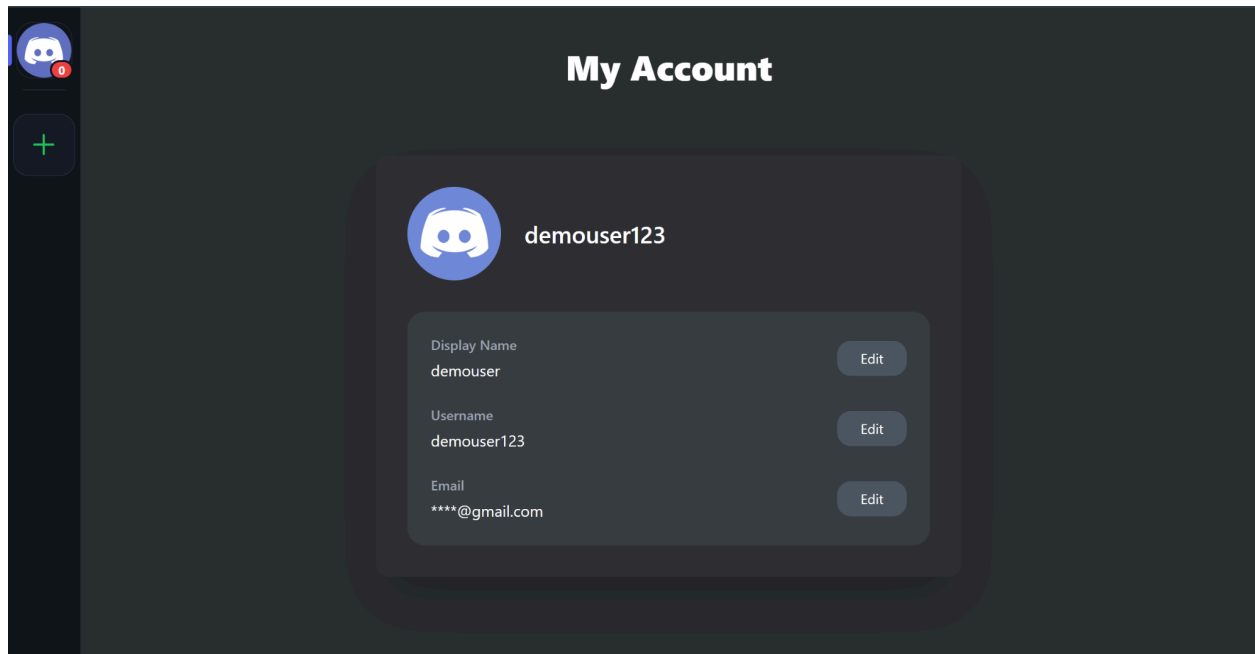
Private Chat (Direct Messaging):



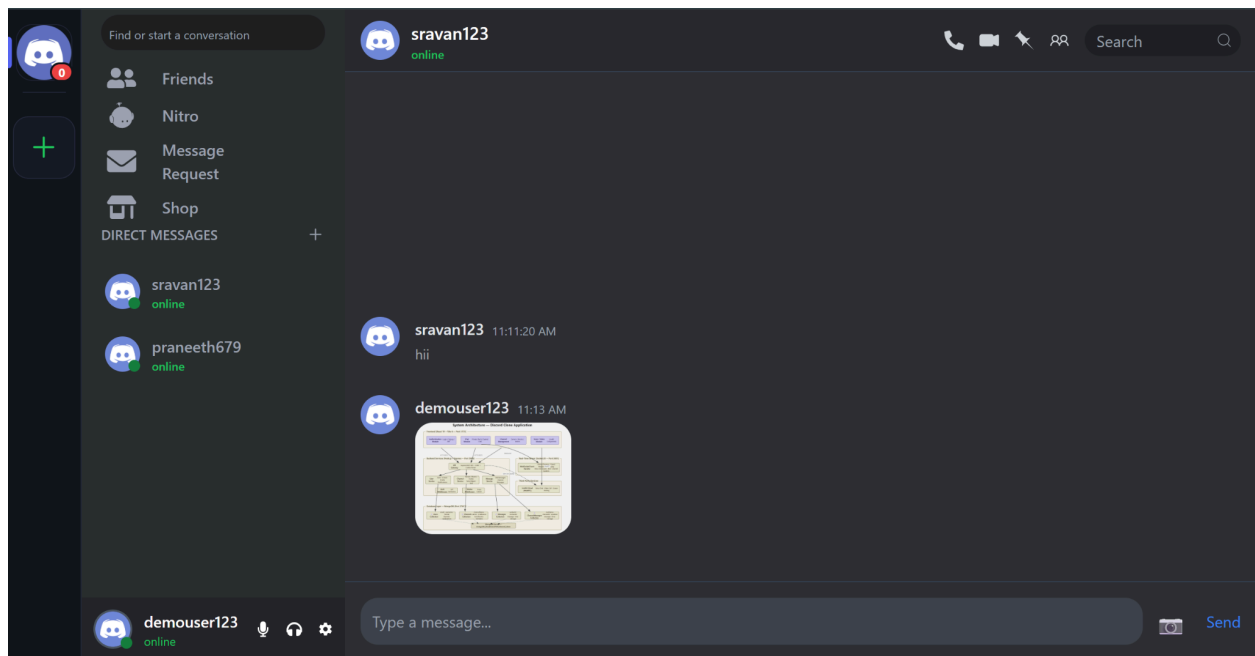
Home Page — Online Friends:



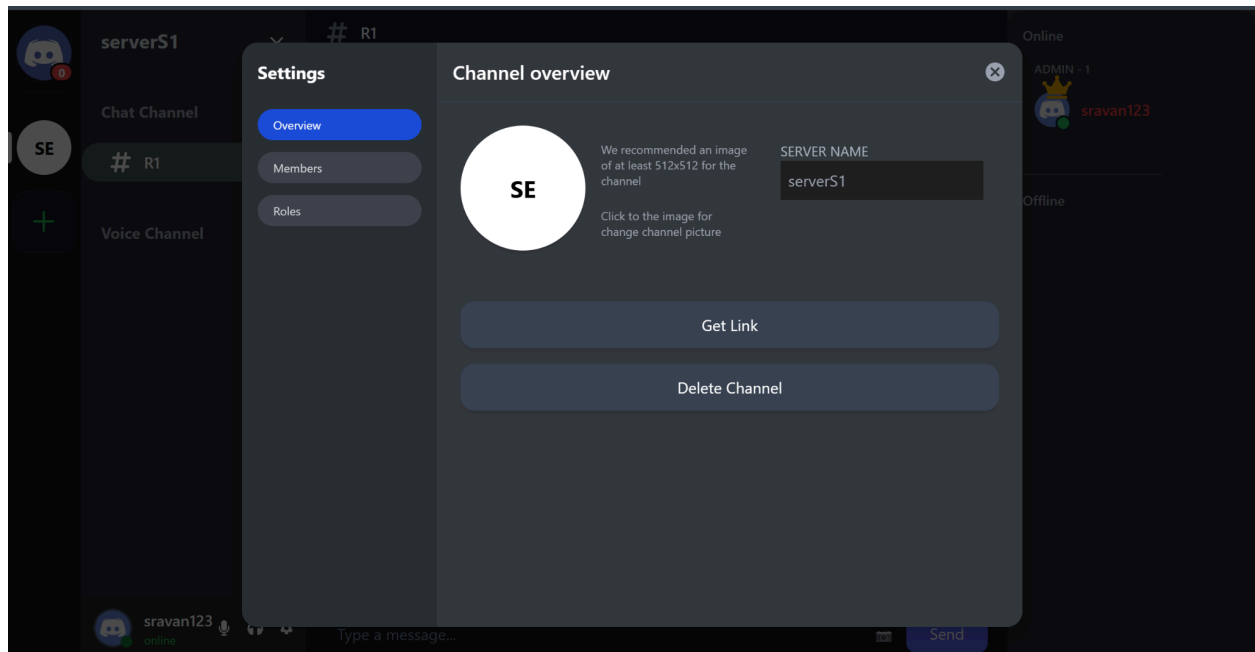
User Profile Page:



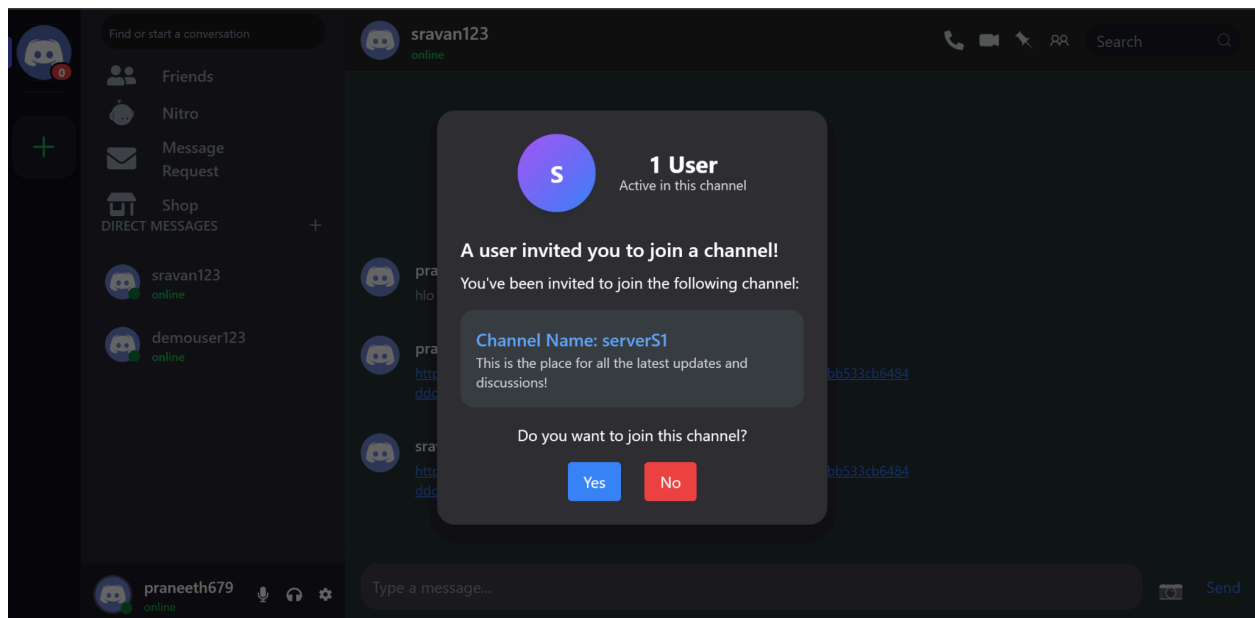
Private Chat – Image Sharing:



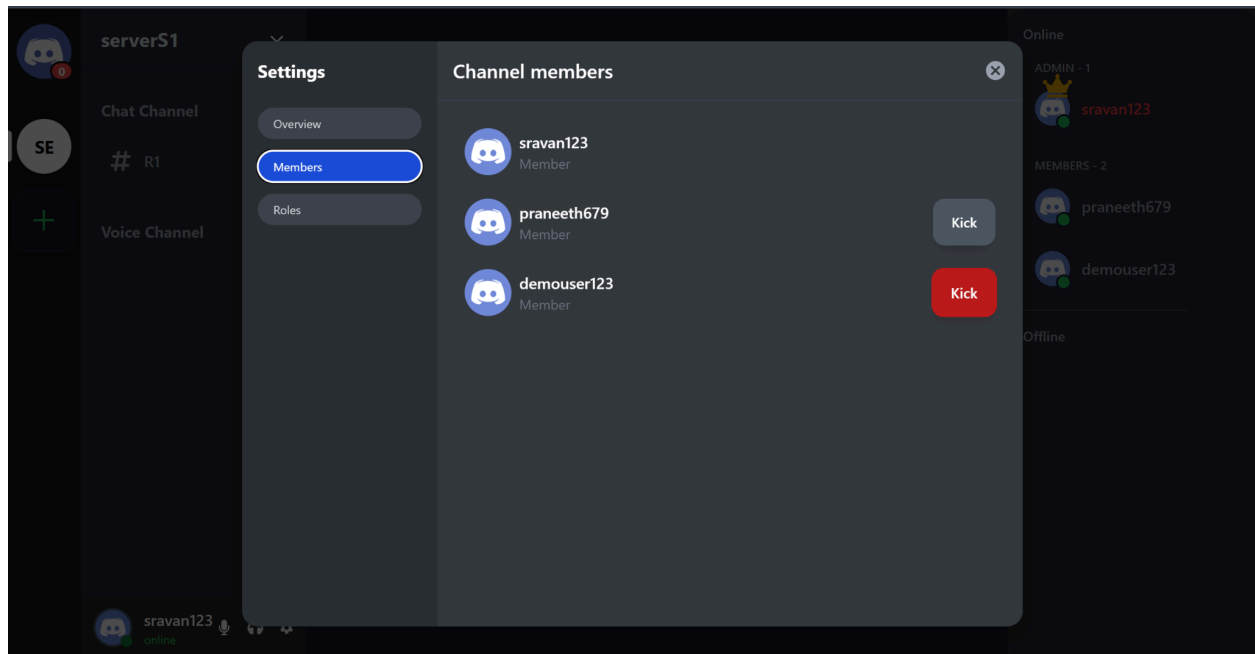
Server Settings – Overview:



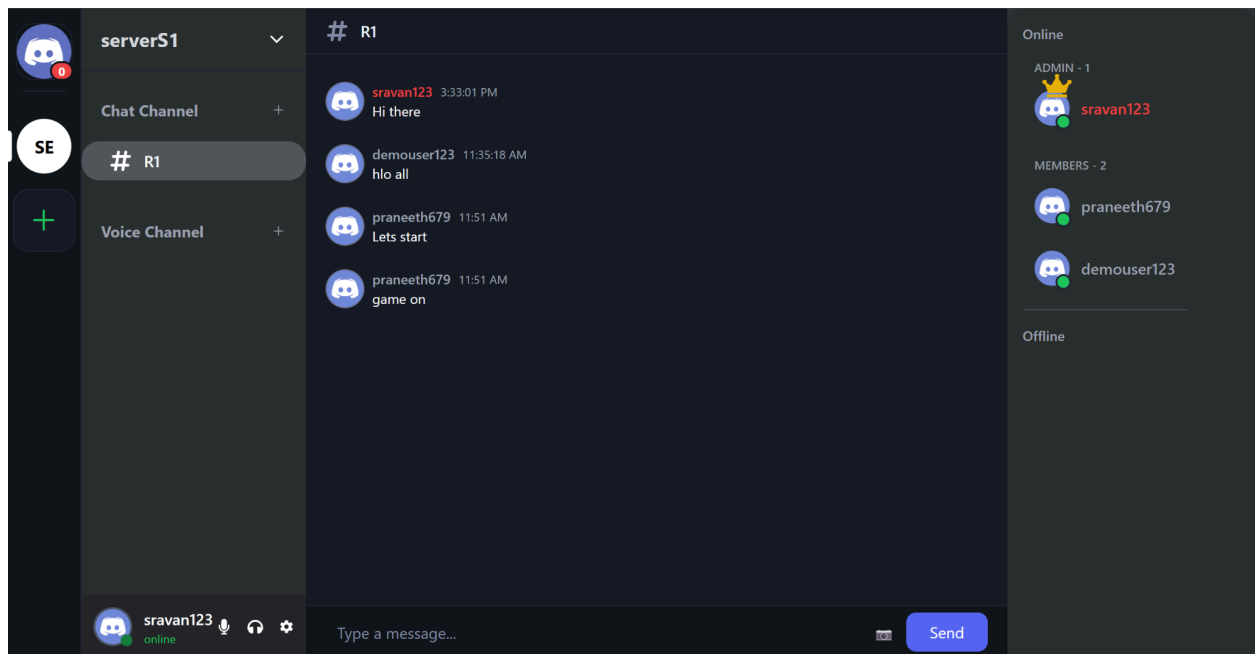
Channel Invitation Interface:



Server Settings – Members Management:



Server Page – Text Channel Chat:



11. Demo Links

Demo Video Link:

https://drive.google.com/file/d/1OJAmikXUkZxnVDewW9_EOxxb04FGF4Vr/view?usp=sharing

Demo Code Link:

https://drive.google.com/drive/u/0/folders/1YN9r5XJlGqHAFSZYD_0ShIYOoO8IYJA